

中国科学技术大学

本科毕业论文



GPU 并行化稀疏 Cholesky 重分解

作者姓名：	计勃帆
学 号：	PB22010389
专 业：	信息与计算科学
导 师：	陈仁杰 教授
完成时间：	2026 年 4 月 21 日

中国科学技术大学本科毕业论文学术诚信承诺书

本人郑重承诺：所呈交的本科毕业论文，是本人在导师指导下进行研究工作所取得的成果。除已特别加以标注和致谢的地方外，论文中不包含任何他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的贡献均已在论文中作了明确的说明。如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文(设计)等违纪行为。

作者签名：_____

签字日期：_____

摘要

稀疏 Cholesky 重分解是在原稀疏对称正定矩阵仅发生局部修改时，复用已有分解结果以高效获得新分解的关键算法，在网格参数化、物理仿真等计算机图形学应用中具有重要意义。然而，现有方法大多针对 CPU 架构设计，在处理大规模问题时受制于并行度与内存带宽，难以充分发挥现代 GPU 的计算潜力。本文拟面向 GPU 并行架构，研究并实现一种高并行度、高加速比的稀疏 Cholesky 重分解算法。

稀疏 Cholesky 重分解通常包含符号分析与数值计算两个阶段，其中符号分析（包括填充减少排序和符号分解）往往占据主要计算成本。填充减少排序需要寻找合适的置换矩阵，使经相合变换后的分解因子尽可能稀疏；符号分解则通过构建消元树、识别超节点，为后续数值分解提供依赖关系和并行调度信息。

由于填充减少排序问题本身为 NP 困难，现有工作多采用启发式策略，并在 CPU 上取得了较成熟的效果，但在 GPU 上仍存在并行度不足、访存不规则等瓶颈。为此，本文基于 k 叉树结构重新设计符号分析与重分解流程，在图划分与消元树构建阶段显式挖掘层次化并行性，从而适配 GPU 上的数值计算算法实现大规模稀疏 Cholesky 重分解的高效实现。本方法在保证分解质量的前提下，显著缩短重分解时间，为大规模图形仿真和几何处理提供高性能基础算法。

关键词：稀疏 Cholesky 重分解；填充减少排序；k 叉树，GPU 并行计算

ABSTRACT

Sparse Cholesky refactorization is a key algorithm for efficiently updating the factorization of a sparse symmetric positive definite matrix when only local modifications occur, and plays an important role in computer graphics applications such as mesh parameterization and physical simulation. However, most existing methods are designed for CPU architectures and are limited by parallelism and memory bandwidth when handling large-scale problems, preventing it from exploiting the computational power of modern GPUs. This paper targets GPU-parallel architectures and investigates a highly parallel sparse Cholesky refactorization algorithm that achieves substantial speedups.

Sparse Cholesky refactorization typically consists of two stages: symbolic analysis and numerical computation. Symbolic analysis (including fill-reducing ordering and symbolic factorization) often dominates the overall computational cost. Fill-reducing ordering seeks an appropriate permutation matrix so that the factor after congruence transformation remains as sparse as possible, while symbolic factorization constructs the elimination tree and identifies supernodes to provide dependency and scheduling information for subsequent numerical factorization.

Since the fill-reducing ordering problem is NP-hard, existing work commonly adopts heuristic strategies that have achieved mature performance on CPUs but still suffer from limited parallelism and irregular memory access on GPUs. To address these issues, this paper redesigns the symbolic analysis and refactorization pipeline based on a k-ary tree structure, explicitly exploiting hierarchical parallelism in graph partitioning and elimination tree construction so as to better match GPU-oriented numerical algorithms for large-scale sparse Cholesky refactorization. Under comparable factorization quality, the proposed method significantly reduces refactorization time and thus provides a high-performance foundational algorithm for large-scale graphical simulation and geometric processing.

Key Words: Sparse Cholesky refactorization; Fill-reducing ordering; k-ary tree; GPU parallel computing

目 录

第一章 问题简介	4
第一节 问题背景	4
第二节 问题描述	5
第三节 研究目标	6
第四节 文章概览	7
第二章 相关研究	9
第一节 AMD 算法	9
一、从 MD 到 AMD 的思想演进	9
二、AMD 算法的实现流程	10
第二节 Metis 图分割	11
一、METIS 顶点分割	12
二、METIS k-way 分割	13
第三节 ND 算法	14
第四节 Parth Cholesky 重分解	14
第五节 消元树	15
一、消元树的定义	15
二、关键性质与作用	15
第三章 核心算法	17
第一节 数据结构	17
第二节 初始化	17
第三节 二次重排序	18
第四章 算法评估	20
第一节 运行环境	20
一、硬件配置和操作系统	20
二、依赖库版本	21
第二节 测试数据说明	21

第三节 运行时间测试	22
第四节 置换结果可视化	23
第五节 算法分析	23
第五章 总结与展望	25
第一节 工作总结	25
第二节 未来展望	25
参考文献	27

插图和附表清单

图 4.1	cathead 网格运行时间	22
图 4.2	igea 网格运行时间	22
图 4.3	igea 网格运行时间	22
图 4.4	igea 网格置换结果可视化	23
表 4.1	硬件配置和操作系统	20
表 4.2	主要依赖库及工具版本	21
表 4.3	测试网格参数	21

第一章 问题简介

第一节 问题背景

在现代计算机图形学与科学计算领域，大规模稀疏线性方程组 ($Ax = b$) 的求解是众多核心算法的计算瓶颈。特别是在网格参数化、三维几何处理以及基于有限元方法的物理仿真等应用中，系统矩阵 A 通常表现为高度稀疏的对称正定矩阵。为了保证求解的稳定性和精度，直接求解法（如稀疏 Cholesky 分解）被广泛采用。然而，随着三维模型分辨率的不断提高和物理仿真场景的日益复杂，系统矩阵的维度往往达到百万甚至千万级别，单次从头进行矩阵分解的计算成本极其高昂。

在实际的交互式图形应用或时间积分步进的仿真过程中，系统矩阵往往并非在每一步都发生全局剧变。例如，在网格的局部交互式编辑、拓扑的局部细分或简化、以及物理引擎中由于局部碰撞响应或材质非线性导致的刚度矩阵更新中，原稀疏矩阵 A 往往只发生局部修改，即只有极少数非零元素的值或结构发生变化。在这种动态演化的场景下，如果每次更新后都将矩阵视为全新矩阵并重新进行完整的 Cholesky 分解将会造成巨大的计算资源浪费。在这样的背景下，稀疏 Cholesky 重分解技术应运而生，其核心思想是通过追踪局部修改在消元过程中的传播路径，最大程度地复用已有的分解结果，仅对受影响的局部数据进行更新，从而大幅提升连续求解的效率。

尽管稀疏 Cholesky 重分解在理论与算法层面上已有较深入的研究，但现有的主流实现与优化方案大多是针对传统的 CPU 架构设计的。稀疏矩阵分解的固有特性——即高度不规则的内存访问模式、复杂的内部数据依赖关系（如消元树的拓扑结构）以及细粒度的分支跳转——使得这些算法在 CPU 的串行或多核粗粒度并行模型下能够取得较好的成熟度。然而，面对日益庞大的图形仿真需求，CPU 在并行计算核心数和内存带宽上的物理瓶颈日益凸显，已难以满足实时或近实时的大规模计算需求。

与此同时，GPU 凭借其海量的计算核心和极高的显存带宽，已经成为高性能计算领域的主力平台。然而，将传统的稀疏 Cholesky 重分解算法直接移植到 GPU 上却面临着巨大的鸿沟。重分解流程中极为关键的符号分析阶段（包含寻找保持分解因子稀疏性的填充减少排序，以及构建消元树和识别超节点的符号

分解), 不仅属于 NP 困难问题而高度依赖启发式策略, 而且其计算图表现出极强的不规则性和动态性。这种特性与 GPU 偏好规则内存合并访问和 SIMT 的架构设计背道而驰, 导致 GPU 的海量线程难以被充分“喂饱”, 实际并行度严重不足。

综上所述, 如何在原稀疏对称正定矩阵仅发生局部修改的普遍场景下, 打破传统 CPU 算法的思维定势, 面向 GPU 大规模并行架构重新设计高并行度、高访存效率的稀疏 Cholesky 重分解算法, 释放现代硬件的计算潜力, 已成为突破大规模图形仿真与几何处理性能瓶颈的一项迫切且具有重要应用价值的研究课题。

第二节 问题描述

在正式探讨本文的算法设计之前, 首先对稀疏 Cholesky 重分解问题进行数学形式化描述, 并剖析其在现代 GPU 架构下面临的核心技术挑战。

给定一个大规模稀疏对称正定矩阵 $A \in \mathbb{R}^{n \times n}$ (通常由有限元离散或三角网格结构派生而来), 其标准的稀疏 Cholesky 分解可表示为:

$$PAP^T = LL^T \quad (1.1)$$

其中, P 为用于减少分解过程中产生填充元 (Fill-ins) 的置换矩阵 (Permutation Matrix), L 为对角线元素大于零的下三角矩阵。当仿真场景或几何模型发生局部变化时, 原矩阵 A 发生局部修改, 更新为新矩阵 $\tilde{A} = A + \Delta A$ 。此处 ΔA 通常是一个稀疏度极高的低秩更新矩阵或局部拓扑变化矩阵。重分解的核心目标是在已知 P 、 L 及其依赖结构的前提下, 快速且高效地计算出新的分解形式:

$$\tilde{P}\tilde{A}\tilde{P}^T = \tilde{L}\tilde{L}^T \quad (1.2)$$

完整的重分解流程通常包含两个主要阶段: 符号分析 (Symbolic Analysis) 与数值计算 (Numerical Factorization)。符号分析进一步分为填充减少排序 (求取置换矩阵 $\tilde{P}$) 和符号分解 (预测 $\tilde{L}$ 的非零元结构、构建消元树并识别超节点)。由于数值计算阶段通常可以通过高度优化的稠密线性代数库 (如 cuBLAS) 在 GPU 上获得极高的执行效率, 符号分析阶段往往暴露出严重的性能短板, 成为制约整体重分解效率的核心瓶颈。

针对现代 GPU 的 SIMT 架构, 实现高效的稀疏 Cholesky 重分解面临以下几个严峻的问题与挑战

- **启发式贪心策略带来的强序列依赖:** 经典的排序算法如 AMD 主要基于贪心策略, 在每一步选择当前度数最小的节点进行模拟消元, 并动态更新其

邻接节点的度数。这种“消元-更新”过程具有极强的序列数据依赖，下一步的操作必须严格等待前一步图拓扑更新的完成。这种内在的串行性使得 GPU 的海量并发线程无法被有效调度，导致严重的计算资源闲置。

- **图划分与多级粗化中的不规则访存：**为了提高并行度，目前更具潜力的排序方法是基于嵌套解剖（Nested Dissection, ND）的图划分算法。然而，ND 算法需要经历多级图粗化、初始划分和解粗化细化过程。由于稀疏矩阵对应的图拓扑高度不规则，节点度数分布不均，导致在 GPU 上执行图遍历和匹配时，线程块之间负载极度失衡；同时，频繁的间接寻址引发了大量非合并显存访问，严重消耗了 GPU 的访存带宽。
- **局部修改下排序结果的增量更新困难：**在重分解场景下，矩阵仅发生局部修改 ΔA 。如果采用全局重新排序，不仅计算代价极为高昂，还会导致前后两次分解的消元树结构完全不同，使得原有的数值因子无法复用。如何精准识别局部拓扑变化对图结构的影响范围，在避免全局图锁定的前提下，利用 GPU 的并发能力对受影响的局部区域进行并行的增量排序与结构合并，是目前算法设计中的一大盲区。

综上所述，本文所要解决的核心问题是：如何打破传统 CPU 上启发式排序算法的序列瓶颈与不规则访存限制，针对原矩阵仅发生局部修改的特性，设计一套完全适配 GPU SIMT 架构特征、显式挖掘图划分与树形结构层次化并行性的填充减少排序与重分解机制。

第三节 研究目标

针对前文所述的传统启发式排序算法在 GPU 上面临的强序列依赖、访存不规则以及局部更新困难等挑战，本文的总体研究目标是：提出并实现一种完全面向 GPU SIMT 架构的高效稀疏 Cholesky 重分解算法。为了打破现有算法的并行瓶颈，本文的核心创新与研究目标集中在重构填充减少排序的计算范式上。

传统基于嵌套解剖（Nested Dissection, ND）的图划分排序方法，通常采用递归的分层二叉树分解。这种自顶向下的多级二叉结构存在两个致命弱点：一是分解初期的顶层节点极少，无法提供足够的独立任务来饱和 GPU 的海量流处理器；二是深层递归引入了极高的全局同步开销和控制流分化。为此，本文提出将传统的逐层二叉树分解重构为平铺的单层 k 叉树分解。

围绕这一核心思想，本文的具体研究目标为：放弃深层递归的二叉划分策

略，转而设计一种高效的单层多路图划分算法，将全局稀疏矩阵拓扑直接分解为 k 个大致均衡的独立子图，构造单层 k 叉树；并且在重分解场景下，利用 k 叉树扁平且分支独立的几何特性，设计一种增量更新机制，当发生局部拓扑修改时，算法能够将其影响精确约束在 k 叉树的特定子分支内，GPU 只需对受影响的少数分支进行局部的重新划分与排序，而无需锁定全局图结构，从而最大化地复用历史分解因子并极大地降低重分解时间。

第四节 文章概览

本文的后续章节安排如下：

第二章相关研究：系统回顾了与稀疏 Cholesky 重分解及图排序相关的经典算法与前沿工作。首先介绍了基于贪心启发式策略的近似最小度（AMD）算法及其思想演进；随后详细阐述了 METIS 图分割库的核心原理，包括顶点分割与 k -way 分割算法；接着介绍了基于递归二分策略的嵌套解剖（ND）算法；随后引出了稀疏矩阵求解中的核心图论数据结构——消元树，探讨了其在符号分析、内存预分配及并行计算调度中的关键作用；最后分析了现有的面向动态稀疏结构的 Parth Cholesky 重分解方法，并总结了现有方法在高度并行化上面临的局限性。

第三章核心算法：详细阐述了本文提出的面向 SIMT 架构的单层 k 叉划分与增量重排序算法。本章首先定义了算法所需的核心数据结构（如 *DOFToNode* 映射与切割顶点标记 *IsCutVertex*）；随后介绍了利用 METIS 进行初始 k -way 划分与全局初次重排序的初始化流程；最后重点讲解了在新增局部边的场景下，如何通过精准识别受影响子图并将其分类（第一类与第二类子图），从而执行高并行度的局部重分解与局部 AMD 调整的二次重排序机制。

第四章算法评估：对本文提出的核心算法进行了全面的实验验证。本章首先说明了实验的软硬件运行环境及依赖库配置，并介绍了用于测试的三个不同规模的典型网格模型（*cathead*, *ra*, *igea*）；随后，通过改变叉数 k 与新增边数量，详细对比了初始化阶段与二次重排序阶段的运行时间，验证了增量策略的高效性；最后，通过置换矩阵的可视化（Spy 图），直观地展示了本算法在保持分块对角结构及降低填充元方面的正确性与预期效果。

第五章总结与展望：对全文的研究工作进行了系统性总结，重申了单层 k 叉图划分与增量重排序机制在打破传统序列依赖、提升并行潜力方面的核心贡献。

同时，指出了当前工作的局限性，并对未来在 GPU 架构上的全面并行化移植、与数值计算阶段的无缝对接，以及支持更复杂的拓扑修改场景（如增加或删除自由度）等后续研究方向进行了展望。

第二章 相关研究

第一节 AMD 算法

在稀疏矩阵的填充减少排序中，最小度（Minimum Degree, MD）算法是历史上最经典的贪心启发式算法。近似最小度（Approximate Minimum Degree, AMD）算法正是在 MD 算法的基础上演进而来，通过在理论模型和数据结构上的创新，极大地提升了排序的计算效率。

一、从 MD 到 AMD 的思想演进

MD 算法的核心思想基于高斯消元过程的图论刻画。对于稀疏对称正定矩阵 A ，其非零元结构可等价表示为一个无向图 $G = (V, E)$ 。在消元过程中，消除一个节点 v 意味着将其从图中移除，并将其所有相邻的未消元节点两两相连，形成一个完全子图。这些新增加的边即对应矩阵分解中产生的填充元。MD 算法的策略是在每一步均选择当前图中度数最小的节点进行消元，以此局部最小化每一步引入的填充元数量。

然而，随着消元的进行，原图中会不断生成密集的团。显式地向图中添加这些边并精确维护每一个未消元节点的准确度数，需要耗费极高的内存与时间开销。为了克服这一理论与计算瓶颈，Amestoy、Davis 和 Duff 提出了 AMD 算法。

AMD 算法的核心思想是放弃对节点精确度数的昂贵计算，转而计算一个易于获取的度数上限，将其作为“近似度数”来指导贪心选择。为了高效计算和维护这一近似度数，AMD 引入了商图（Quotient Graph）模型。在商图中，节点被严格划分为两类：未消元的原始节点称为“变量（Variables）”，而已消元的节点及其产生的团被合并为一个整体，称为“元素（Elements）”。一个变量的近似度数，可以通过统计与之相邻的所有元素包含的变量总数，并扣除不可避免的重叠部分来快速计算。这种将完全图隐式表达为集合操作的思想，彻底避免了在消元过程中显式添加填充元边的庞大开销。

二、AMD 算法的实现流程

AMD 算法的完整执行流程主要围绕商图的构建、元素的合并以及近似度数的动态更新展开，其具体实现伪代码如下

算法 2.1 AMD Algorithm

Data: Undirected graph of a sparse symmetric matrix $G = (V, E)$

Result: Permutation vector P

```

1  $P \leftarrow \emptyset$ ;
2 Initialize the quotient graph;
3 foreach  $v \in V$  do
4   | Compute the initial degree and set it as the approximate degree  $d(v)$ ;
5 end
6 while there exists an uneliminated variable in  $V$  do
7   | Find  $p \in V$  with minimum approximate degree  $d(p)$ ;
8   | Append  $p$  to the end of  $P$ ;
9   | Remove  $p$  from the set of uneliminated variables  $V$ ;
10  | Convert the pivot  $p$  into a new quotient-graph element  $E_p$ ;
11  | Let  $\mathcal{N}(p)$  be the set of uneliminated variables adjacent to  $p$ ;
12  | foreach existing element  $E_q$  adjacent to  $E_p$  do
13    | Merge all variables in  $E_q$  into  $E_p$ ;
14    | Delete the absorbed redundant element  $E_q$  from the quotient graph;
15  | end
16  | foreach uneliminated variable  $v$  related to  $E_p$  do
17    | Recompute and update the upper bound of the approximate degree
18    |    $d(v)$  based on the updated quotient graph;
19  | end
20 end
21 return  $P$ ;

```

第二节 Metis 图分割

METIS 是一种广泛使用的图分割和稀疏矩阵填充减少排序的软件库。其核心思想是基于多级图分割范式 (Multilevel Graph Partitioning Paradigm)，旨在在保证分割质量的同时大幅降低计算复杂度。多级图分割通常包含三个主要阶段：

1. 粗化阶段 (Coarsening Phase)：将原始图逐渐收缩为一系列规模越来越小的粗图，直到图的规模足够小。

2. 初始分割阶段 (Initial Partitioning Phase)：在最粗的图上计算初始分割。由于图的规模非常小，这一步可以采用相对复杂的启发式算法而不会消耗太多时间。

3. 反粗化与优化阶段 (Uncoarsening and Refinement Phase)：将初始分割逐级映射回原始图，并在每一级上使用局部优化算法 (如 FM 或 KL 算法) 对分割边界进行调整，以进一步减小割边数或顶点分割集的大小。

METIS 主要提供两种主流的图分割策略：顶点分割和 k-way 分割。下面分别对其功能和算法流程进行详细阐述。

一、METIS 顶点分割

顶点分割的主要目标是将图的顶点集划分为两个互不相交的子集，同时寻找一个最小的顶点分割集（Vertex Separator）使得去除这些顶点后图被分成两个近似等大的连通分量。在 METIS 中，顶点分割常作为递归二分的基础步骤，用于生成稀疏矩阵的嵌套解剖（Nested Dissection）排序。

算法 2.2 METIS Vertex Separator Algorithm

Data: Original graph $G = (V, E)$

Result: Partition $P = \{V_1, V_2\}$ and separator set S

```

1  $G_0 \leftarrow G$ ;
2  $i \leftarrow 0$ ;
3 while  $|V(G_i)|$  is large do
4   Find a matching in  $G_i$  using Heavy-Edge Matching;
5   Contract matched vertices to form a coarser graph  $G_{i+1}$ ;
6    $i \leftarrow i + 1$ ;
7 end
8 Compute an initial 2-way partition  $P_i$  of the coarsest graph  $G_i$  using a
   heuristic algorithm;
9 while  $i > 0$  do
10   $i \leftarrow i - 1$ ;
11  Project partition  $P_{i+1}$  back to the finer graph  $G_i$  to obtain an initial
     partition  $P_i$ ;
12  Refine  $P_i$  using FM or KL to reduce the cut cost;
13 end
14 return  $P_0$  and  $S$ ;
```

二、METIS k -way 分割

当需要将图划分为 k 个子图 ($k > 2$) 时, 虽然可以通过递归调用上述的二路分割来实现, 但随着 k 的增大, 递归二分法不仅速度变慢, 且由于每次二分都缺乏全局视角, 容易导致最终的分割质量下降。为了解决这个问题, METIS 引入了 k -way 分割算法。该方法在多级框架下, 直接在最粗的图上生成 k 个分区, 并在反粗化阶段全局协同地对 k 个分区的边界进行调整。这种方法极大地降低了算法的时间复杂度, 并且通常能够获得割边数更少的分割结果。

算法 2.3 METIS k -way partitioning algorithm

Data: Original graph $G = (V, E)$, number of partitions k

Result: k -way partitioning $P = \{V_1, V_2, \dots, V_k\}$

```

1  $G_0 \leftarrow G$ ;
2  $i \leftarrow 0$ ;
3 while  $|V_i| > c \cdot k$  do
4   | Generate a coarser graph  $G_{i+1}$ ;
5   |  $i \leftarrow i + 1$ ;
6 end
7 Compute an initial  $k$ -way partition  $P_i$  on  $G_i$  using recursive bisection or
  another multiway strategy;
8 while  $i > 0$  do
9   |  $i \leftarrow i - 1$ ;
10  | Project partition  $P_{i+1}$  back to the finer graph  $G_i$  to obtain  $P_i$ ;
11  | if  $|V(G_i)|$  is large then
12    | Randomly traverse boundary vertices and try moving each to a
13    | neighboring partition to maximize gain;
14  | else
15    | Apply a more fine-grained greedy local refinement strategy;
16  | end
17 end
18 return  $P_0$ ;

```

第三节 ND 算法

Nested Dissection (ND, 嵌套剖分) 算法是一种面向稀疏矩阵/稀疏图的重排序方法, 其基本思想为利用 Metis 顶点分割算法, 寻找一个顶点分割集 $S \subset V$, 将图在移除 S 后划分为两个互不相连的子图 V_1, V_2 。在消元意义下, 若将分割集对应的变量延后消元, 则填充主要被限制在各个子域内部; 相反, 若过早消元分割集, 则可能在不同子域之间引入大量新的耦合边, 导致显著的填充增长。因此, ND 通过递归地“先子域、后分分割集”(即后序遍历)的排序策略, 实现对填充的抑制, 并形成适合分治与并行的层次结构。

给定图 $G = (V, E)$, ND 算法递归地产生一个全局的变量排序, 其实现流程如下

1. 若当前顶点集合规模 $|V|$ 小于预设阈值, 则直接返回一个基准排序作为递归终止条件。否则, 在当前图上寻找一个分割集 S , 使得删除 S 后剩余顶点集合 $V \setminus S$ 被划分为两个连通分量 V_1, V_2 , 并且连通分量之间不存在边连接。
2. 对每个诱导子图 $G[V_i]$ 递归调用 ND, 分别得到子域内的局部排序 p_i 。在得到所有子域排序后, 将它们按任意固定顺序拼接, 并将分割集 S 对应的顶点排列追加到末尾, 从而得到当前层的排序结果 $p = [p_1, p_2, S]$ 。
3. 递归返回到顶层后, 最终获得全局置换 p 。

第四节 Parth Cholesky 重分解

Parth 是一种基于 ND 实现的面向动态稀疏结构的稀疏 Cholesky 重分解方法。其核心目标是: 当矩阵非零模式仅在局部发生变化时, 尽可能复用上一时刻的填充降低排序与符号分析结果, 只对受影响的局部区域进行增量更新, 从而减少每一步的符号阶段成本, 并最终降低整个 Cholesky 求解时间。

Parth 将稀疏矩阵对应为图, 而动态仿真或局部重网格引起的非零结构变化, 在图上通常表现为局部子图的边/点增删。Parth 通过分层图分解来组织矩阵结构, 使其能够在结构更新时定位“变化传播”的范围。Parth 的实现流程如下

1. 调用 ND 算法, 计算初始置换向量和图的二叉分解树;
2. 当检测到稀疏结构改变(例如新增/删除自由度、局部耦合关系变化)时, 先在分解结构上进行同步/标记, 确定哪些子图块受到影响;
3. 仅对这些受影响块执行必要的重排序与符号更新; 对未受影响的大部分区域直接复用上一帧的排序与符号信息。这一步的关键在于对于祖先发生改变

的节点，只需对子树重新分解，而无需对子节点重排，从而实现算法剪枝。

第五节 消元树

消元树 (Elimination Tree) 是稀疏矩阵分解 (尤其是稀疏 Cholesky 分解) 中最为核心的图论数据结构之一。它深刻揭示了高斯消元过程中各个变量 (或矩阵列) 之间的数据依赖关系, 是稀疏矩阵求解中符号分析、内存预分配以及并行计算调度的重要理论基础。

一、消元树的定义

给定一个 $n \times n$ 的稀疏对称正定矩阵 A , 经过某种填充减少排序 (如 AMD 或 ND) 后, 其 Cholesky 分解因子为 L (即 $A = LL^T$)。消元树 T 是一个包含 n 个节点 (对应矩阵的 n 列或变量) 的树状图结构。

在消元树中, 节点 j 的父节点 $p(j)$ 被严格定义为因子矩阵 L 第 j 列对角线以下第一个非零元素所在的行索引, 即:

$$p(j) = \min\{i > j \mid L_{ij} \neq 0\}$$

如果第 j 列对角线以下所有元素均为零, 则节点 j 没有父节点, 它将成为消元树 (或森林) 的根节点。

二、关键性质与作用

消元树在稀疏矩阵数值计算中扮演着不可替代的角色, 其核心性质和作用主要体现在以下几个方面:

1. **数据依赖与并行性分析:** 在 Cholesky 分解的高斯消元过程中, 第 j 步的消元操作只会直接影响到它的父节点 $p(j)$, 并沿着消元树向上传递。这意味着, 消元树中任何不存在祖孙关系的两个节点 (例如位于完全不同分支的子树节点) 之间没有任何数据依赖。这种性质使得属于不同分支的矩阵列可以被分配到不同的处理器上, 完全独立且并行地进行数值消元。

2. **符号分解与内存分配:** 消元树的拓扑结构完全决定了因子矩阵 L 的非零元分布。根据图论性质, 原矩阵 A 中的任意非零元 A_{ij} ($i > j$) 都意味着在消元树中节点 j 必定是节点 i 的后代。通过对消元树的后序遍历, 求解器可以在不执行任何实际浮点运算 (即数值分解) 的情况下, 精确预测出 L 中每一列的非零元素个数及其位置。这一步骤被称为符号分解, 它允许程序在进行耗时的数值计

算前提前、精确地分配所需的连续内存空间。

3. 与排序算法的紧密联系：前面介绍的 AMD 和 ND 等重排序算法，其本质除了减少填充元之外，也在重塑消元树的形态。特别是 ND 算法，由于其递归二分的特性，天然会生成一棵“短而宽”的消元树。位于树底部的众多浅层分支对应于被分割的局部子域，这些分支可以被高度并行地处理；而靠近树根的节点则对应于各个分割集。这种结构极大地提升了稀疏矩阵分解的并行效率。

第三章 核心算法

我们只考虑自由度不变，添加边的情形。这是因为对于减少边或减少自由度的情形，我们可以直接复用之前的置换向量；而对于增加自由度的情形，只需将新增边涉及的子图标记为第二类（见第三节）即可。

第一节 数据结构

与 ND 算法的分层二叉树分解不同，本文采用单层 k 叉划分。分解树中每个叶子节点代表一个子图，根节点仅作为形式化的分割集，不具备真实物理含义。后续增量算法只需维护“顶点属于哪个分区”以及“哪些顶点与跨分区边相关”等信息，因此不显式存储每个子图的顶点集合。

为实现后续算法，在数据结构中保存：

- k ：整型变量，分解树的叉数。
- $DOFToNode$ ：长度为 $|V|$ 的整型数组，代表从全局自由度到其所属子图编号的映射。给定顶点 u ，其所在子图编号为 $DOFToNode[u]$ 。该映射可以代替显式表示子图顶点集合。
- $IsCutVertex$ ：长度为 $|V|$ 的布尔数组，用于标记顶点是否为跨分区边的端点。

第二节 初始化

初始化阶段的目标是：对原始图进行一次 k 路划分以建立 $DOFToNode$ 映射并计算初始切割顶点标记，随后对全图执行一次 AMD 重排序得到初始置换向量。

给定图 $G = (V, E)$ 及其稀疏邻接存储 (Mp, Mi) ，初始化流程如下：

1. **确定分区数**：令 $k \leftarrow \min(k, |V|)$ ，防止子图数大于自由度。
2. **准备编号映射**：构造数组 $assigned_DOF$ ，在初次分解时令

$$assigned_DOF[i] = i, \quad i = 0, 1, \dots, |V| - 1,$$

即局部编号与全局编号一致，便于将划分结果回填到全局索引空间。

3. **METIS k -way 划分**：调用 `METIS_PartGraphKway` 对图进行 k 路划分，

得到每个局部顶点的分区编号 $part[i]$ 。

4. **构造 $DOFToNode$** : 对每个局部顶点 i , 其全局编号为 $g = assigned_DOF[i]$, 回填

$$DOFToNode[g] \leftarrow part[i].$$

5. **计算切割顶点 $IsCutVertex$** : 对每个顶点 u , 若存在邻居 v 满足

$$DOFToNode[u] \neq DOFToNode[v],$$

则置 $IsCutVertex[u] = 1$, 否则为 0。

6. **初次 AMD 重排序**: 对全图调用 `amd_order` 得到置换向量 `mesh_perm`。
若图无边 ($NNZ = 0$), 则置换向量为恒等置换。

初始化完成后, $DOFToNode$ 、 $IsCutVertex$ 以及初始 `mesh_perm` 将作为新增边后的二次重排序的基础。

第三节 二次重排序

当图中新增边后, 若每次都对全图重新分解与重排序将带来较大开销。为此采用增量二次重排序策略: 首先识别新增边, 再判断这些新增边影响的分区集合, 并对受影响区域执行局部重分解与局部 AMD 更新。

设旧图与新图分别为 G_{old} 与 G_{new} , 其邻接存储分别为 (Mp_{old}, Mi_{old}) 与 (Mp_{new}, Mi_{new}) 。二次重排序流程如下:

1. **识别新增边**: 比较 G_{old} 与 G_{new} , 计算新增边集合

$$E^+ = E_{new} \setminus E_{old}.$$

若 $E^+ = \emptyset$ 则直接返回, 不进行更新。

2. **对子图分类**: 对每条新增边 $(u, v) \in E^+$, 令 $p_u = DOFToNode[u]$, $p_v = DOFToNode[v]$, 并维护分区标记数组 `mark[p]`:

- 若 $p_u \neq p_v$ (即新增边跨分区), 或 $IsCutVertex[u] = 1$ 或 $IsCutVertex[v] = 1$, 则将该边相连的两个分区标为第一类。
- 否则 (边完全在同一分区内部且端点不是切割顶点), 则将该分区标为第二类。
- 第一类标记可以覆盖第二类标记。

特别地, 若新增边的端点为切割顶点, 则实现中还会将该切割顶点在新图中直接相连的邻居所在分区也标为第一类, 用于捕获扰动可能扩散到多个

分区的情况。

3. 第一类子图：统一重分解 + 重排序：

- (a) 收集所有第一类分区的顶点集合，形成诱导子图 G_1 ；
- (b) 从新图中提取 G_1 的邻接结构；
- (c) 若第一类分区个数为 $k_1 > 1$ 且子图规模足够大，则对 G_1 调用 `METIS_PartGraphKway` 做 k_1 路重划分，并将结果回填到全局 `DOFToNode`（分区编号取自原第一类分区 ID 数组，以保持编号空间一致）；
- (d) 对 G_1 执行 AMD 得到子图置换，并仅在全局置换向量 `mesh_perm` 中更新属于 G_1 顶点的相对顺序。

4. 第二类子图：逐分区局部 AMD：对每个被标记为第二类的分区 p ：

- (a) 收集该分区顶点集合并提取其诱导子图；
- (b) 在该子图上执行 AMD；
- (c) 将 AMD 结果回填到全局 `mesh_perm` 中该分区顶点对应的位置，从而仅改变分区内部的相对顺序。

5. 更新切割顶点标记：二次重排序完成后，基于更新后的 `DOFToNode` 在新图上重新计算 `IsCutVertex`：对每个顶点 u ，若存在邻居 v 满足 `DOFToNode[u] $\neq$ DOFToNode[v]`，则 `IsCutVertex[u] = 1`，否则为 0。

通过上述“分类 + 局部处理”的策略，算法避免了对全图进行完全重分解与完全重排序：只有跨分区新增边或涉及切割顶点的区域会触发较重的 k -way 处理，其余仅需分区内 AMD 调整，从而提升增量场景下的整体效率。

第四章 算法评估

第一节 运行环境

一、硬件配置和操作系统

表 4.1 硬件配置和操作系统

Table 4.1 Hardware Configuration and Operating System

配置项	参数
处理器架构	x86_64
CPU 型号	12th Gen Intel(R) Core(TM) i7-12700H
物理核心数	10
逻辑线程数	20
每核线程数	2
L1d 缓存	480 KiB (10 instances)
L1i 缓存	320 KiB (10 instances)
L2 缓存	12.5 MiB (10 instances)
L3 缓存	24 MiB (1 instance)
内存容量	7.6 GiB
操作系统 (WSL 发行版)	Ubuntu
操作系统版本	24.04.3 LTS
Linux 内核版本 (WSL)	6.6.87.2-microsoft-standard-WSL2
虚拟化环境	WSL (Hypervisor vendor: Microsoft)
编译器版本	GCC 13.3.0

二、依赖库版本

表 4.2 主要依赖库及工具版本

Table 4.2 Versions of Main Dependencies and Tools

依赖项	版本
C++ Standard	C++17
CMake	3.28.3
Eigen3	3.4.0
OpenMP	5.1
METIS	5.1.0
SuiteSparse	7.6.1
MATIO	1.5.26

第二节 测试数据说明

用于生成 Laplace 矩阵的网格相关参数如下表所示

表 4.3 测试网格参数

Table 4.3 Testing Mesh Parameters

网格名称	顶点数	边数
cathead	131	378
ra	10044	30132
igea	134345	403029

由于 k-way 分解是随机算法，分解结果不同会导致运行时间波动，后续将做多次测试，结果取平均。

第三节 运行时间测试

三个网格在修改 2,5,10 条边后的首次 k-way 分解、首次 amd 重排序和第二次重排序所用的平均时间随叉数 k 的变化折线图如下所示, 其中 cathead, ra, igea 的测试分别为 10000, 1000, 1000。

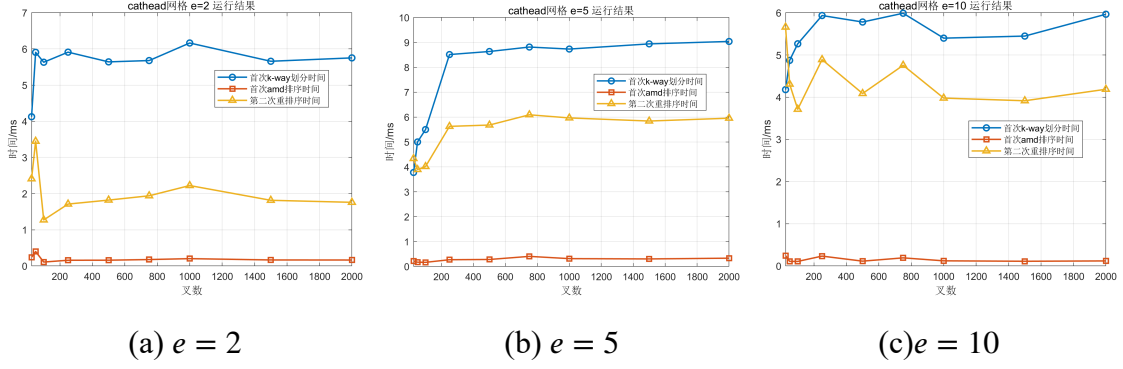


图 4.1 cathead 网络运行时间

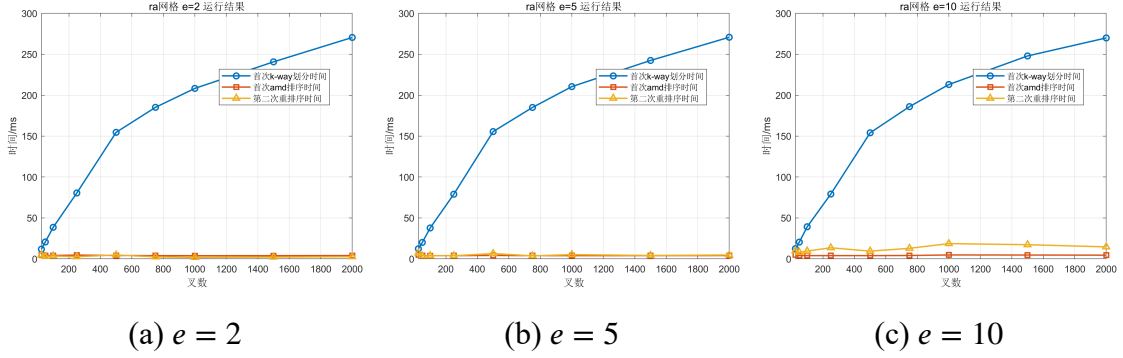


图 4.2 igea 网络运行时间

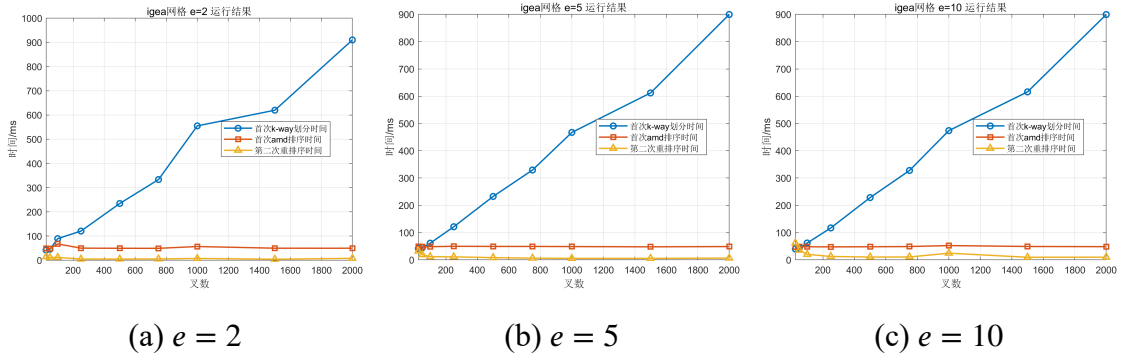


图 4.3 igea 网络运行时间

第四节 置换结果可视化

在填充减少排序中，我们期望的置换结果为

$$P^T A P = \begin{bmatrix} \Sigma & \alpha \\ \alpha^T & C \end{bmatrix}$$

其中 Σ 是分块对角矩阵（对应每个分区）， α 和 C 是稠密矩阵（对应分区之间的邻接关系）。下面展示 igea 网格添加 2 条边的置换结果（由于置换结果有一定随机性，这里只能展示运行中的部分结果）

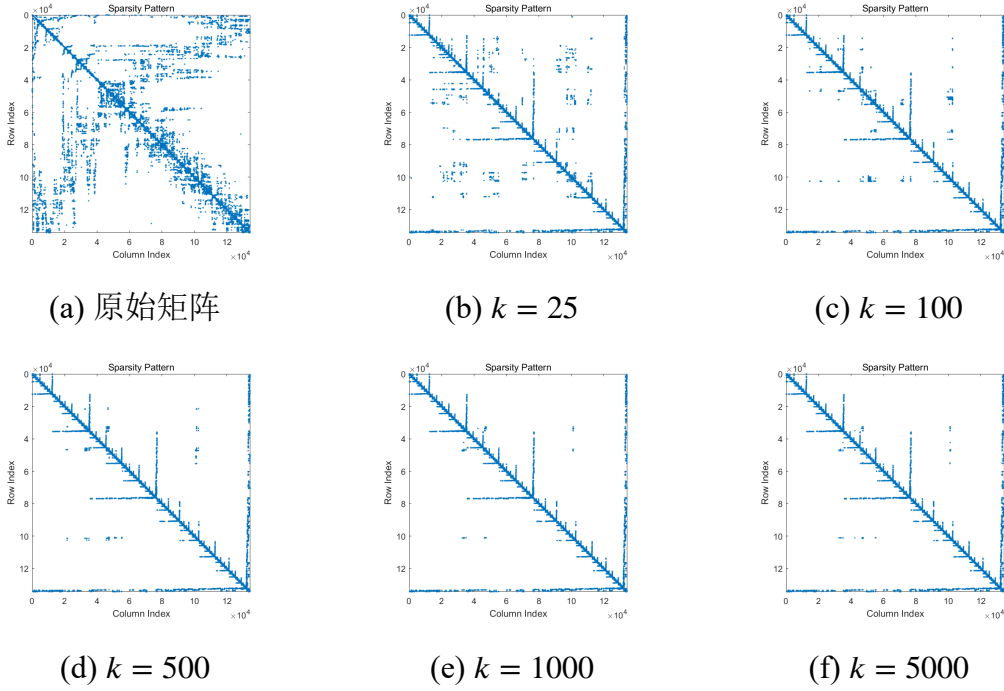


图 4.4 igea 网格置换结果可视化

第五节 算法分析

基于上述实验结果，可得出如下结论：

- 针对小规模网格（顶点数量级小于 10^5 ），直接执行 AMD 重排序的时间开销显著低于构建 k 叉分解树的成本。
- 针对大规模网格，随着分解树分支数 k 的增加，初始的 k -way 分解耗时呈现明显上升趋势，但后续的局部重分解时间相应减少，且整体分解质量获得提升。

综上所述，本文所提算法更适用于处理网格规模庞大且边拓扑更新稀疏的场景。此外，针对拓扑结构局部变化较频繁的应用需求，采用更高分支数 k 的分

解树能够有效摊销重组成本，从而获得更优的计算效率。

第五章 总结与展望

第一节 工作总结

本文提出并实现了一种面向 SIMT 架构友好的单层 k 叉图划分与增量重排序算法，对稀疏 Cholesky 重分解中的符号分析阶段进行了重新设计。本文的主要工作总结如下：

- **重构分解拓扑，提出单层 k 叉划分框架：**打破了传统 ND 算法自顶向下的多级二叉树递归模式，引入 METIS 的 k -way 分割算法，将全局稀疏矩阵对应的图拓扑一次性扁平化划分为 k 个近似均衡的独立子图。这种平铺结构不仅大幅减少了全局同步开销，还为后续在 GPU 上的大规模并发调度提供了充足的独立任务。
- **设计基于局部受扰区域的增量重排序机制：**针对实际仿真中矩阵仅发生局部拓扑修改（如新增边）的特性，本文提出了一种高效的增量更新策略。通过引入 *DOFToNode* 映射与切割顶点标记，算法能够快速识别拓扑变化的影响范围，并将子图划分为两类：对于跨分区或涉及切割顶点的第一类子图，执行局部的统一重分解与重排序；对于仅限分区内部的第二类子图，仅执行极轻量级的逐分区局部 AMD 调整。该机制成功避免了全局重排序的巨大开销。
- **算法有效性的实验验证：**本文在多组标准测试网格（cathead, ra, igea）上进行了详细的实验评估。运行时间测试结果表明，增量二次重排序的时间开销显著低于首次全局重排序，并且通过调整叉数 k ，算法在不同规模的模型上均能找到性能平衡点。同时，置换矩阵的可视化结果也直观地证实了本文算法能够成功将原始矩阵转换为具有分块对角结构（各个独立子图）和低秩稠密边界（切割顶点集）的期望形态，有力证明了算法在保持填充减少特性上的正确性。

第二节 未来展望

尽管本文在稀疏矩阵重分解的符号分析阶段取得了阶段性的算法创新，但在将其完全部署为实用的高性能图形计算引擎组件方面，仍有以下几个方向有

待进一步深入探索：

- **GPU 并行架构的全面移植与优化：**目前的实验评估主要在验证单层 k 叉分解算法的逻辑正确性与增量更新的理论效率。下一步的工作重点是将局部重分解和多路独立子图的局部 AMD 排序完全移植到 CUDA 平台上，利用显存合并访问机制和 Warp 级并行策略，彻底释放 GPU 的吞吐量优势。
- **数值计算阶段的无缝对接：**符号分析的核心目的是为数值分解服务。未来需要结合本文生成的特殊形态置换向量，定制化开发基于 GPU 的多级批量 Cholesky 分解核函数，使得符号分析层面的高并发性能能够顺滑地过渡到浮点运算阶段。
- **支持更复杂的拓扑修改场景：**目前本文算法主要针对“自由度不变、增加边”的高频场景进行了深度优化。未来将在此框架基础上，进一步拓展对增加/删除自由度、以及大规模结构剧变等更复杂动态场景的支持，完善增量更新分类策略，使其成为一个完全通用的动态稀疏线性方程组求解框架。

参 考 文 献

- [1] Chan R H, Greif C, O' Leary D P. Methods for modifying matrix factorizations (With P. E. gill, W. murray and M. A. Saunders)[M]//Milestones In Matrix Computation: Selected Works Of Gene H. Golub, With Commentaries. Oxford University Press, 2007.
- [2] Bartels R, Kaufman L. Cholesky factor updating techniques for rank 2 matrix modifications[J]. SIAM Journal on Matrix Analysis and Applications, 1989, 10 (4): 557-592.
- [3] Yannakakis M. Computing the minimum fill-in is NP-complete[J]. SIAM Journal on Algebraic Discrete Methods, 1981, 2(1): 77-79.
- [4] Amestoy P R, Davis T A, Duff I S. Algorithm 837: AMD, an approximate minimum degree ordering algorithm[J]. ACM Trans. Math. Softw., 2004, 30(3): 381-388.
- [5] Zarebavani B, M. Kaufman D, I. W. Levin D, et al. Adaptive algebraic reuse of reordering in cholesky factorizations with dynamic sparsity patterns[J]. ACM Trans. Graph., 2025, 44(4).
- [6] Liu J W. A compact row storage scheme for cholesky factors using elimination trees[J]. ACM Trans. Math. Softw., 1986, 12(2): 127-148.
- [7] Davis T A, Hager W W. Modifying a sparse cholesky factorization[J]. SIAM Journal on Matrix Analysis and Applications, 1999, 20(3): 606-627.
- [8] Davis T A, Hager W W. Dynamic supernodes in sparse cholesky Update/Downdate and triangular solves[J]. ACM Trans. Math. Softw., 2009, 35 (4).
- [9] Karsavuran M O, Ng E G, Peyton B W. GPU accelerated sparse cholesky factorization[C]//SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2024: 703-707.
- [10] Herholz P, Alexa M. Factor once: reusing cholesky factorizations on sub-meshes [J]. ACM Trans. Graph., 2018, 37(6).
- [11] Herholz P, Sorkine-Hornung O. Sparse cholesky updates for interactive mesh

- parameterization[J]. ACM Trans. Graph., 2020, 39(6).
- [12] Li J, Liu T, Kavan L, et al. Interactive cutting and tearing in projective dynamics with progressive cholesky updates[J]. ACM Trans. Graph., 2021, 40(6).
- [13] Yuan C. Fast sparse matrix reordering on GPU for cholesky based solvers[D]. University of California, Davis, 2024.
- [14] Karypis G, Kumar V. Multilevelk-way partitioning scheme for irregular graphs [J]. Journal of Parallel and Distributed Computing, 1998, 48(1): 96-129.
- [15] Karypis G, Kumar V. A fast and high quality multilevel scheme for partitioning irregular graphs[J]. SIAM Journal on scientific Computing, 1998, 20(1): 359-392.